

Системне Програмування

З використанням мови програмування **Rust.**
Fundamentals. Char. Encodings

Slicing

```
let hello : &str = "hello";  
dbg!(&hello[0..2]); // "he"  
dbg!(&hello[0..4]); // "hell"
```

Slicing

```
let hello : &str = "Привіт";  
dbg!(&hello[0..2]); // П
```

Slicing

```
let hello : &str = "Привіт";  
dbg!(&hello[0..4]); // Пр
```

Slicing

```
let hello : &str = "Привіт";  
dbg!(&hello[0..3]); // panic
```

Ітерація по символам

```
let s5: String = "hello".to_string();  
for c: char in s5.chars() {  
    print!("{}", c);  
}  
// h e l l o
```

Ітерація по байтам

```
let s5: String = "Hello".to_string();  
for c: u8 in s5.bytes() {  
    print!("{}", c);  
}  
// 72 101 108 108 111
```

ASCII Table

Dec	Hx	Oct	Char	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr
0	0	000	NUL (null)	32	20	040	 	Space	64	40	100	@	@	96	60	140	`	`
1	1	001	SOH (start of heading)	33	21	041	!	!	65	41	101	A	A	97	61	141	a	a
2	2	002	STX (start of text)	34	22	042	"	"	66	42	102	B	B	98	62	142	b	b
3	3	003	ETX (end of text)	35	23	043	#	#	67	43	103	C	C	99	63	143	c	c
4	4	004	EOT (end of transmission)	36	24	044	$	\$	68	44	104	D	D	100	64	144	d	d
5	5	005	ENQ (enquiry)	37	25	045	%	%	69	45	105	E	E	101	65	145	e	e
6	6	006	ACK (acknowledge)	38	26	046	&	&	70	46	106	F	F	102	66	146	f	f
7	7	007	BEL (bell)	39	27	047	'	'	71	47	107	G	G	103	67	147	g	g
8	8	010	BS (backspace)	40	28	050	(	(	72	48	110	H	H	104	68	150	h	h
9	9	011	TAB (horizontal tab)	41	29	051	)	)	73	49	111	I	I	105	69	151	i	i
10	A	012	LF (NL line feed, new line)	42	2A	052	*	*	74	4A	112	J	J	106	6A	152	j	j
11	B	013	VT (vertical tab)	43	2B	053	+	+	75	4B	113	K	K	107	6B	153	k	k
12	C	014	FF (NP form feed, new page)	44	2C	054	,	,	76	4C	114	L	L	108	6C	154	l	l
13	D	015	CR (carriage return)	45	2D	055	-	-	77	4D	115	M	M	109	6D	155	m	m
14	E	016	SO (shift out)	46	2E	056	.	.	78	4E	116	N	N	110	6E	156	n	n
15	F	017	SI (shift in)	47	2F	057	/	/	79	4F	117	O	O	111	6F	157	o	o
16	10	020	DLE (data link escape)	48	30	060	0	0	80	50	120	P	P	112	70	160	p	p
17	11	021	DC1 (device control 1)	49	31	061	1	1	81	51	121	Q	Q	113	71	161	q	q
18	12	022	DC2 (device control 2)	50	32	062	2	2	82	52	122	R	R	114	72	162	r	r
19	13	023	DC3 (device control 3)	51	33	063	3	3	83	53	123	S	S	115	73	163	s	s
20	14	024	DC4 (device control 4)	52	34	064	4	4	84	54	124	T	T	116	74	164	t	t
21	15	025	NAK (negative acknowledge)	53	35	065	5	5	85	55	125	U	U	117	75	165	u	u
22	16	026	SYN (synchronous idle)	54	36	066	6	6	86	56	126	V	V	118	76	166	v	v
23	17	027	ETB (end of trans. block)	55	37	067	7	7	87	57	127	W	W	119	77	167	w	w
24	18	030	CAN (cancel)	56	38	070	8	8	88	58	130	X	X	120	78	170	x	x
25	19	031	EM (end of medium)	57	39	071	9	9	89	59	131	Y	Y	121	79	171	y	y
26	1A	032	SUB (substitute)	58	3A	072	:	:	90	5A	132	Z	Z	122	7A	172	z	z
27	1B	033	ESC (escape)	59	3B	073	;	;	91	5B	133	[	[	123	7B	173	{	{
28	1C	034	FS (file separator)	60	3C	074	<	<	92	5C	134	\	\	124	7C	174	|	
29	1D	035	GS (group separator)	61	3D	075	=	=	93	5D	135	]	]	125	7D	175	}	}
30	1E	036	RS (record separator)	62	3E	076	>	>	94	5E	136	^	^	126	7E	176	~	~
31	1F	037	US (unit separator)	63	3F	077	?	?	95	5F	137	_	_	127	7F	177		DEL

Ітерація по символам

```
let s5: String = "Привіт".to_string();  
for c: char in s5.chars() {  
    print!("{}", c);  
}  
// П р и в і т
```

Ітерація по байтам

```
let s5: String = "Привіт".to_string();  
for c: u8 in s5.bytes() {  
    print!("{}", c);  
}  
// 208 159 209 128 208 184 208 178 209 150 209 130
```

Ітерація по байтам

```
let s5: String = "Привіт".to_string();  
for c: u8 in s5.bytes() {  
    print!("{}", c);  
}  
// 208 159 209 128 208 184 208 178 209 150 209 130
```

```
let s1: Bytes = hello[0..2].bytes();  
for b: u8 in s1 {  
    print!("{:08b} ", b);  
}  
// 11010000 10011111
```

UTF8 encoding

```
let s1: Bytes = hello[0..2].bytes();
for b: u8 in s1 {
    print!("{:08b} ", b);
}
// 11010000 10011111
// ^^^      ^^
//   10000   11111
//
//   1000011111
```

UTF8 encoding

```
11010000 10011111
```

UTF8 encoding

<https://www.metadas.com/utf8/cyrillic/>

CHARACTER	NUMBER	HEX	UNICODE	ENTITY
А	1040	0410	U+0410	А
Б	1041	0411	U+0411	Б
В	1042	0412	U+0412	В
Г	1043	0413	U+0413	Г
Д	1044	0414	U+0414	Д
Е	1045	0415	U+0415	Е
Ж	1046	0416	U+0416	Ж
З	1047	0417	U+0417	З
И	1048	0418	U+0418	И
Й	1049	0419	U+0419	Й
К	1050	041A	U+041A	К
Л	1051	041B	U+041B	Л
М	1052	041C	U+041C	М
Н	1053	041D	U+041D	Н
О	1054	041E	U+041E	О
П	1055	041F	U+041F	П

CHARACTER	NUMBER	HEX	UNICODE	ENTITY
В	1074	0432	U+0432	в
Г	1075	0433	U+0433	г
Д	1076	0434	U+0434	д
е	1077	0435	U+0435	е
Ж	1078	0436	U+0436	ж
з	1079	0437	U+0437	з
и	1080	0438	U+0438	и
Й	1081	0439	U+0439	й
К	1082	043A	U+043A	к
Л	1083	043B	U+043B	л
М	1084	043C	U+043C	м
Н	1085	043D	U+043D	н
О	1086	043E	U+043E	о
п	1087	043F	U+043F	п
р	1088	0440	U+0440	р

UTF8 encoding

Bytes	Pattern	Meaning
1	0xxxxxxx	ASCII
2	110xxxxx 10xxxxxx	2-byte sequence
3	1110xxxx 10xxxxxx 10xxxxxx	3-byte sequence
4	11110xxx 10xxxxxx 10xxxxxx 10xxxxxx	4-byte sequence

UTF8 encoding

p = 11010001 10000000

UTF8 encoding

"नमस्ते"

UTF8 encoding

"नमस्ते"

```
dbg!(&chars);  
// ['न', 'म', 'स्', 'ते']
```

UTF8 encoding

"नमस्ते"

```
dbg!(&bytes);  
// [224, 164, 168, 224, 164, 174, 224, 164, 184,  
// -----  
// 224, 165, 141, 224, 164, 164, 224, 165, 135]  
// -----
```

UTF8 encoding

```
dbg!(&bytes);  
// [224, 164, 168, 224, 164, 174, 224, 164, 184,  
// -----  
// 224, 165, 141, 224, 164, 164, 224, 165, 135]  
// -----
```

```
// 11100000 10100100 10101000
```

UTF8 encoding

11100000	10100100	10101000
^^^	^^	^^
0000	100100	101000

UTF8 encoding

```
11100000 10100100 10101000
^^^  ^^  ^^
      0000  100100  101000
      0000100100101000
```

UTF8 encoding

Hex:

0x0928

Unicode:

U+0928

Decimal:

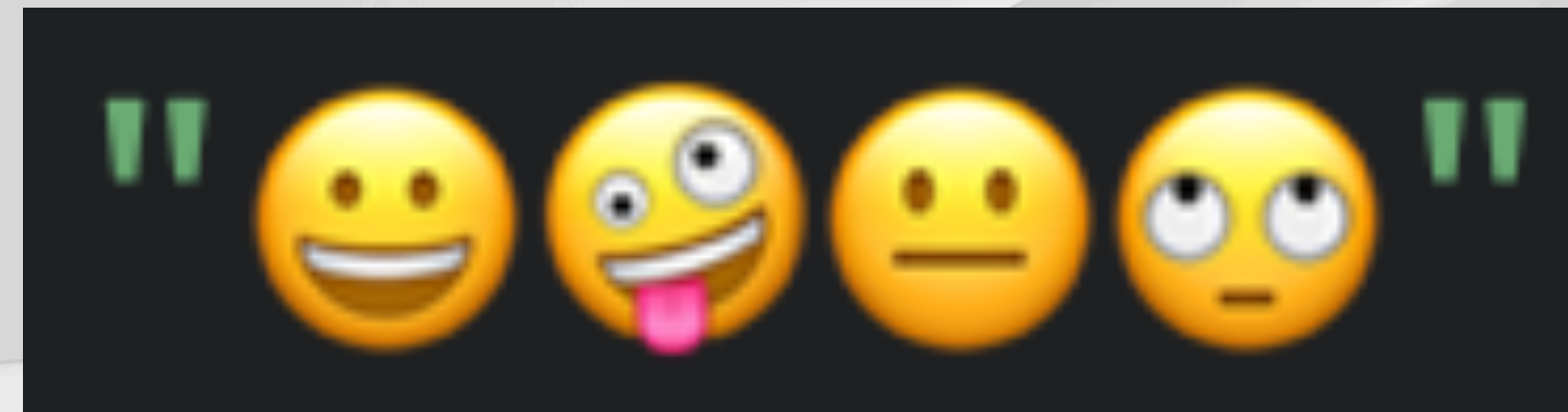
2344

न	2344	0928	U+0928
---	------	------	--------

UTF8 encoding

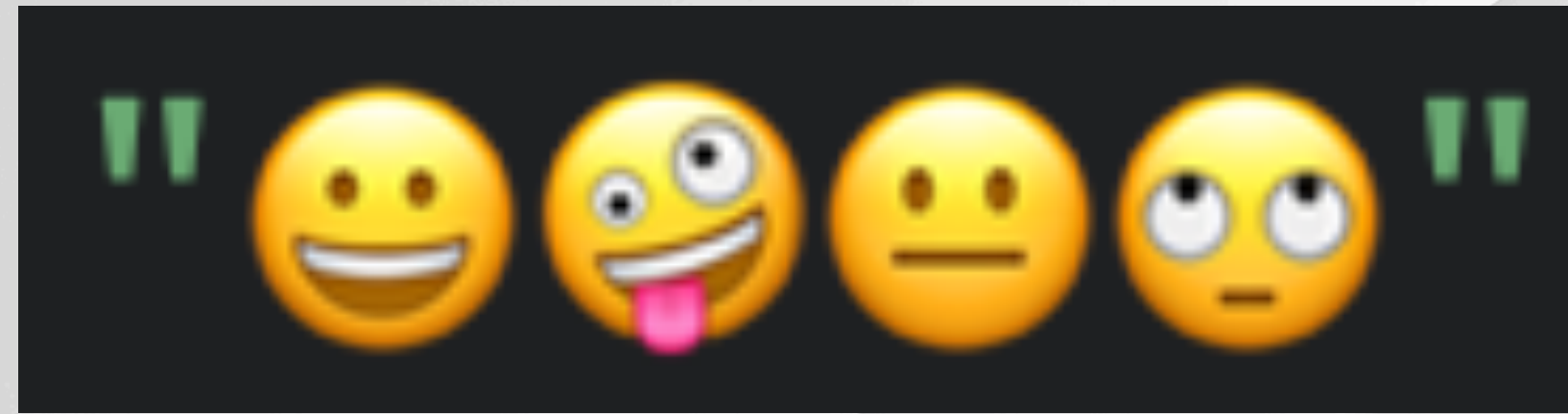


UTF8 encoding



```
// bytes
//
// 240, 159, 152, 128, 240, 159, 164, 170,
// -----
// 240, 159, 152, 144, 240, 159, 153, 132,
// -----
```

UTF8 encoding



```
// bytes
//
// 240, 159, 152, 128, 240, 159, 164, 170,
// -----
// 240, 159, 152, 144, 240, 159, 153, 132,
// -----
```

```
11110000 10011111 10011000 10000000
```

UTF8 encoding

```
11110000 10011111 10011000 10000000
```

UTF8 encoding

11110000 10011111 10011000 10000000

Combine:

00001111101100000000

Binary → hex:

1F600



GRINNING FACE

The dark side. Char

```
let c1: char = 'z';  
let c2: char = 'я';  
let c3: char = '你';  
let c4: char = '😊';
```

```
println!("{}", size_of_val(&c1)); // 4  
println!("{}", size_of_val(&c2)); // 4  
println!("{}", size_of_val(&c3)); // 4  
println!("{}", size_of_val(&c4)); // 4
```

UTF-8

- може бути 1 байт
- може бути 2 байти
- може бути 3 байти
- може бути 4 байти

але... в Rust `size_of(char)` = 4 байти

```
let hello : String = String::from(s: "pa");
```

```
let hello : String = String::from(s: "pa");  
let chars: Chars = hello.chars();  
let bytes: Bytes = hello.bytes();  
dbg!(&hello); // "pa"  
dbg!(chars.count()); // 2  
dbg!(bytes.count()); // 3  
dbg!(hello.len()); // 3
```

```
let hello : &str = "नमस्ते";
```

```
let hello: &str = "नमस्ते";  
// ['न', 'म', 'स', '्', 'त', 'े']  
// ["न", "म", "स्", "ते"]
```

```
let chars: Chars = hello.chars();  
let bytes: Bytes = hello.bytes();
```

```
dbg!(hello); // "नमस\u{94d}त\u{947}"  
dbg!(&chars); // ['न', 'म', 'स', '\u{94d}', 'त', '\u{947}']  
// ['न', 'म', 'स', '्', 'त', 'े']
```

```
dbg!(&bytes); // [224, 164, 168, 224, 164, 174, 224, 164, 184,  
// -----
```

224, 165, 141,	224, 164, 164,	224, 165, 135]
-----	-----	-----

```
let hello:&str = "😄😜😐😬";  
println!("{}", hello); // 😄😜😐😬  
  
hello.chars().for_each(|c:char| print!("{}", c));  
println!(); // 😄, 😜, 😐, 😬,  
  
hello.bytes().for_each(|b:u8| print!("{}", b));  
println!();  
// 240, 159, 152, 128, 240, 159, 164, 170,  
// -----  
// 240, 159, 152, 144, 240, 159, 153, 132,  
// -----  
println!("{}", hello.len()) // 16
```

UTF8 encoding

Bytes	Pattern	Meaning
1	0xxxxxxx	ASCII
2	110xxxxx 10xxxxxx	2-byte sequence
3	1110xxxx 10xxxxxx 10xxxxxx	3-byte sequence
4	11110xxx 10xxxxxx 10xxxxxx 10xxxxxx	4-byte sequence

```
let s:char = 'a';  
println!("{:?}", bytes(s)); // [97]  
let s:char = 'щ';  
println!("{:?}", bytes(s)); // [208, 169]  
let s:char = '你';  
println!("{:?}", bytes(s)); // [228, 189, 160]  
let s:char = '💡';  
println!("{:?}", bytes(s)); // [240, 159, 146, 161]
```